

Tide retrospective: multi-index affine-bilinear covariance mirror

Daniel Murfet

7 May 2026

Abstract

A 101-line mirror tide. The multi-index analogue of I3’s affine-bilinear covariance family,¹ ported line-for-line to the new namespace. Three public theorems (one-sided affine reductions on each slot, and the headline two-sided identity); same four-witness integrability budget; same proof skeleton; same `funext + ring` bridge for the `Pi.add-vs-single-lambda` mismatch. The exercise is strictly operational; the load-bearing observation is that the I3 abstraction was at exactly the right level for the port to be mechanical. No GPT-5.5 Pro consult; mirror-shape, proceeding-without-GPT path. 0 `sorry`.

1 Setting

I3 (the affine-bilinear specialisation tide, also 7 May) landed three theorems in the 1D `gibbsCov` namespace demonstrating the I2 algebra works in practice. The multi-index covariance on $(\iota \rightarrow \mathbb{R})$ has the parallel-mirror algebra, also added by I2; the natural next step is to mirror the I3 theorems into the multi-index namespace, giving the affine-bilinear identity for both 1D and multi-index Gibbs measures.

The motivation is downstream: the 2D pure-quartic file currently manually expands the affine-cov bashes that would collapse to one-liners under the new helpers. The cleanup proper is the P1 follow-up; this tide just lands the algebra.

The candidates and proof skeleton were pre-determined by I3. Per the tide skill’s “proceed-without-GPT” path, established by the L1 sextic-J-function tide for mirror-shaped tides where the candidate is already deliberated and the closed form is already verified, this tide skips the GPT-5.5 Pro consult.

2 The thing formalised

For the multi-index Gibbs measure $\exp(-t \cdot L(w)) dw$ on $\iota \rightarrow \mathbb{R}$ (with ι a finite type) and any pair of scalar observables $\varphi, \psi : (\iota \rightarrow \mathbb{R}) \rightarrow \mathbb{R}$ satisfying the four base integrability conditions, the multi-index Gibbs covariance satisfies the affine-bilinear identity

$$\text{Cov}_t[a_1\varphi + b_1, a_2\psi + b_2] = a_1 \cdot a_2 \cdot \text{Cov}_t[\varphi, \psi] \quad (a_1, b_1, a_2, b_2 \in \mathbb{R}).$$

The Lean signature of the headline², with ι a finite type implicit:

¹The 1D namespace `Laplace.gibbsCov.affine.*`; the multi-index mirror lives in `Laplace.Multi`.

²In `Laplace/Multi/AffineBilinearCov.lean`.

```

theorem gibbsCov_affine_affine
  (L : (ι → ℝ) → ℝ) (t : ℝ) (a₁ b₁ a₂ b₂ : ℝ)
  (φ ψ : (ι → ℝ) → ℝ)
  (h_exp : Integrable (fun w => Real.exp (-(t * L w))))
  (h_φ    : Integrable (fun w => φ w * Real.exp (-(t * L w))))
  (h_ψ    : Integrable (fun w => ψ w * Real.exp (-(t * L w))))
  (h_φψ   : Integrable (fun w => φ w * ψ w * Real.exp (-(t * L w)))) :
  gibbsCov L t (fun w => a₁ * φ w + b₁) (fun w => a₂ * ψ w + b₂)
    = a₁ * a₂ * gibbsCov L t φ ψ

```

The two one-sided helpers (`_affine_left` and `_affine_right`) take the same four hypotheses and reduce a single affine slot to a scalar multiple.

3 Proof strategy

Identical to I3's, with w a function in $\iota \rightarrow \mathbb{R}$ in place of $x \in \mathbb{R}$ and the `Laplace.Multi.gibbsCov*` algebra in place of the 1D version.

The single-slot reduction splits the affine combination via `gibbsCov_add_left`, kills the constant via `gibbsCov_const_left` and `add_zero`, and factors the scalar via `gibbsCov_smul_left`. Integrability witnesses derive from the four base hypotheses by `Integrable.const_mul` and a `simp` `[mul_assoc]` cleanup.

The right-mirror is three lines via two applications of `gibbsCov_symm` sandwiching the left-version.

The headline composes the two one-sided versions with the right-slot mixed integrability witnesses derived via `Integrable.add` through a `funext + ring` bridge, and a final `← mul_assoc` to fold $a_1 \cdot (a_2 \cdot \cdot) = a_1 a_2 \cdot \cdot$.

4 Roadblocks and resolutions

None. The port was line-for-line identical to I3, except for the lambda variable name (w instead of x) and the namespace prefix. The expected friction points (the `Pi.add-vs-single-lambda` mismatch, the `simp` `[mul_assoc]` cleanup, the `Integrable.const_mul + Integrable.add` construction) all worked verbatim from the I3 template.

The exercise lands first try at every step. `lake build` succeeded on the first compile.

The 101-line file size matches I3's 101-line size to the line, an unexpectedly precise match; structurally the two files are isomorphic.

5 What was Mathlib, what was new

Mathlib. `Integrable.const_mul`, `Integrable.add`. Standard ring tactics.

New. Three theorems and zero infrastructure (`gibbsCov_affine_left/right/affine_affine` in `Laplace.Multi`). Everything else was already present from I2's multi-index algebra, the multi-index Gibbs definitions, and I3's proof structure.

6 Lessons

1. **Mirror tides are nearly free when I2-shaped abstractions exist.** The I2 algebra was built explicitly to factor across the 1D and multi-index Gibbs definitions, with parallel-mirror lemma names (`Laplace.gibbsCov_*` and `Laplace.Multi.gibbsCov_*`). Once I3 worked in 1D, the multi-index mirror was three minutes after the tide-log was drafted, plus build verification.
2. **Mirror-shape tides should skip the GPT consult.** The L1 sextic-J-function tide established the precedent; this tide re-confirms it. When the candidate is identical-modulo-namespace to a just-landed tide, the deliberation has already happened in the predecessor; the `tide-pick [in-progress]` `In progress` $\rightarrow$ $\checkmark$ `Closed` cycle still applies, but the Step 2 GPT round is overhead.
3. **The right time to land a mirror tide is right after the predecessor.** I3 landed earlier this session with the proof template fresh in cache; mirroring it now meant zero context-loading cost. The same template a week from now would require re-reading I3's retrospective and the `laplace CLAUDE.md` to recover the `Pi.add-vs-single-lambda` fix. Strike while the iron is hot.

7 Follow-ups

- **2D pure-quartic affine-cov cleanup** (the natural P1 sub-tide that this tide unblocks). Now that `Laplace.Multi.gibbsCov_affine_*` exists, the 2D pure-quartic file's affine-cov bashes can be replaced with one-liners. Estimated ~ 80 line net deletion. Should be a separate tide (separate retrospective; cleaner narrative).
- **Promote the affine helpers to the gibbsCov-algebra blocks.** Both the 1D and multi-index versions now exist as sibling files; after 2–3 consumer sites land for each, fold the helpers into the `gibbsCov-algebra` sections of the corresponding base Gibbs files so they sit alongside their predecessors.
- **Threepoint-side analogue (Q4 in the v4 survey).** Threepoint's `gibbsCov` (with $h \cdot A$ in the potential) has no algebra at all. A port of I2 + I3 to threepoint would unblock κ_3 strict-improvements stacked on the perturbed measure. Bigger; cross-repo.